

# Lab 03: Tele-Communication System

## Implementation Report

Ansh Agrawal (24B0909)  
 Bibaswan Biswas (24B0915) Prasoon Badjatya (24B0916) Saksham Khandelwal (24B0965)

## Contents

|   |                               |   |
|---|-------------------------------|---|
| 1 | Introduction                  | 1 |
| 2 | System Overview               | 1 |
| 3 | Physical Layer and Link Layer | 1 |
| 4 | Optical Symbol Design         | 2 |
| 5 | Framing and Error Control     | 2 |
| 6 | ARQ Protocol                  | 2 |
| 7 | Conclusion                    | 2 |

## 1 Introduction

In the preliminary design done in the first lab, We proposed a two-phone communication system that moves data without Wi-Fi, Bluetooth, or any existing network stack: a **screen-to-camera optical channel** for the forward data path, and a **speaker-to-microphone acoustic channel** for reverse control signalling (ACK/NACK). That report left several parameters open — the symbol alphabet, framing sizes, redundancy scheme, and acoustic encoding. In this report we summarize how those were finalized, and frames the resulting system in terms of the two communication layers it actually implements.

## 2 System Overview

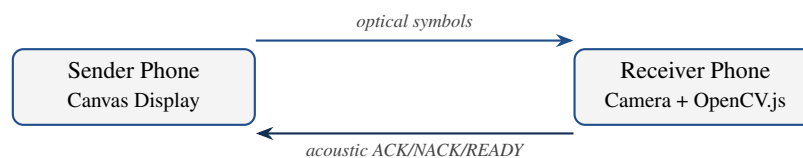


Figure 1: Both directions are paced: the sender waits for a per-symbol tone before advancing, and the receiver waits for a stable clock transition before sampling.

Both phones run identical software and can act as either sender or receiver. The forward (data) direction and reverse (control) direction use physically different media — light and sound — so that they cannot interfere with each other.

## 3 Physical Layer and Link Layer

It is useful to view the system the way a network stack is usually described, split into two layers:

- **Physical layer** — how raw symbols actually move through the environment. On the forward channel we used the canvas display (colour cells + clock cell + finder markers) and the OpenCV.js camera pipeline that detects markers, perspective-corrects the frame, and classifies each cell's colour back into bits. On the reverse channel we used the dual-tone chord generator and the FFT-based tone detector. Neither side has any notion of “messages” — only symbols, colours, and tones.
- **Link layer** — how a reliable, variable-length message is built on top of that unreliable raw symbol stream. This is where framing (SYNC/LENGTH/DATA/END), the Hamming(30,25) error-control code, and the stop-and-wait ARQ protocol are. This layer is responsible for knowing where a frame starts and ends, correcting a single bit error, and deciding when to retransmit.

This separation is why/how the two halves of the system can be developed and debugged independently by us: the physical layer only needs to reliably deliver *some* bits, and the link layer is what turns those bits into a correct, acknowledged message.

## 4 Optical Symbol Design

Each screen frame encodes an 8-bit symbol as a  $2 \times 2$  grid of coloured cells (white/red/green/blue, 2 bits/cell), surrounded by four magenta finder markers (a hue absent from natural scenes, so it is trivial to isolate) and one black/white clock cell that toggles on every new symbol so the receiver knows exactly when to sample.

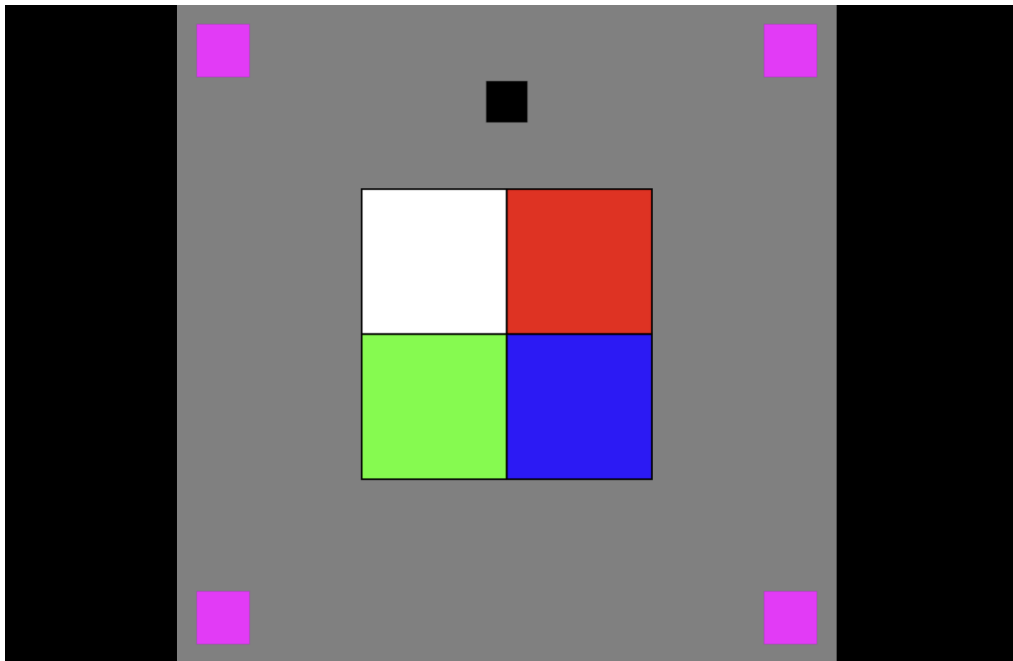


Figure 2: Physical-layer symbol layout on the sender's screen.

## 5 Framing and Error Control

A 48-bit frame (six 8-bit symbols) carries the message:

| Field | SYNC | Hamming codeword | END | pad | Total |
|-------|------|------------------|-----|-----|-------|
| Bits  | 7    | 30               | 4   | 7   | 48    |

SYNC and END are fixed marker patterns bracketing the frame so the receiver can locate it inside a longer, noisy bit stream via a sliding-window search. The 30-bit codeword itself is a **Hamming(30,25)** encoding of a 25-bit data block (5-bit LENGTH + 20-bit zero-padded PAYLOAD): parity bits sit at positions 1, 2, 4, 8, 16, each covering a fixed subset of the other positions. At the receiver, recomputing all five parity checks yields a *syndrome* — zero if nothing changed, or the exact 1-indexed position of a single flipped bit otherwise — which is corrected directly. This gives single-bit error correction with only 5 redundant bits, matching the lab's guarantee of at most one corrupted bit per transmission.

## 6 ARQ Protocol

On top of framing, the link layer runs a **stop-and-wait ARQ**: the sender shows one symbol, waits (bounded) for a per-symbol acoustic ACK, then advances. After all six symbols are shown, a second, frame-level ACK/NACK confirms whether the reassembled frame passed Hamming decoding. A NACK — or a timeout at either level — triggers a full retransmission (without the artificial error, per the lab's guarantee that retries are clean). This two-tier acknowledgement catches both a single missed symbol and a fully corrupted frame, rather than only detecting failure at the end of a transmission.

## 7 Conclusion

The implementation realizes every item the preliminary design left open, organized cleanly into a physical layer (optical/acoustic symbol transport) and a link layer (framing, error correction, ARQ) — the same separation of concerns used in conventional network stacks, applied here to a system built entirely from light and sound.